

Audit complet detaille du code source FSVB Studio

- Date : 2026-04-06
- Application audittee : fsvb-studio-portfolio/portfolio-app
- Nature du livrable : audit complet du code source
- Methode : lecture du code, lecture des scripts SQL, lint local, recoupement avec les preuves Lighthouse existantes, regard tech lead senior sur les bonnes pratiques

1. Resume decisionnel

Verdict global

Le projet a une vraie base produit.

Il ne demande pas une refonte totale.

En revanche, il demande maintenant une reprise plus rigoureuse sur les bonnes pratiques de developpement, car la logique metier critique a grandi plus vite que le cadre technique.

Ce qui est confirme

- la base React + TypeScript + Vite est en place
- le produit couvre deja vitrine, configurateur, pre-session, commande, confirmation et admin
- les risques les plus forts sont concentres sur peu de zones
- les flux critiques melangent encore trop le navigateur, la logique metier et la source de verite
- la maintenabilite commence a baisser fortement sur les gros fichiers

Lecture simple

En langage direct :

1. le produit existe vraiment
2. la dette n'est pas partout
3. les plus gros risques sont localises

- il faut maintenant remettre de l'ordre technique avant d'ajouter beaucoup de nouvelles fonctions

2. Perimetre audite

Code source principal

- `src/App.tsx`
- pages publiques : accueil, solutions, about, contact
- parcours business : configurateur, pre-session, commande, confirmation
- espace admin : login, dashboard, detail commande
- templates publics : landing-solo, restaurant, coach
- socle transverse : `supabase.ts` , services, styles globaux, footer, boutons, modales

Preuves complementaires utilisees

- `npm run lint`
- rapports Lighthouse presents dans `reports/evidence/`
- `package.json` , `tsconfig.app.json` , `eslint.config.js`
- scripts SQL du dossier `sql/`
- contenu du dossier `public/`

3. Indicateurs confirmes

Volume et dette

- `ConfiguratorPage.tsx` : **6963 lignes**
- `FormulairePréSession.tsx` : **2139 lignes**
- `LandingSolo.tsx` : **1033 lignes**
- lint local : **220 problemes**
- dont **197 erreurs** et **23 warnings**
- tests automatises visibles : **aucun script de test dans** `package.json`
- fichiers de test releves dans le projet : **aucun**

Structure applicative

- `App.tsx` importe les grosses pages en statique
- aucun `React.lazy` releve dans `src`
- le widget Crisp est monte globalement depuis `App.tsx`
- `index.html` expose un seul titre global
- `public/` ne contient ni `robots.txt` ni `sitemap.xml`

Socle outillage

- TypeScript est en `strict: true`
- ESLint React + TypeScript est en place
- Vite et build info sont en place
- aucun script `test`, `typecheck` dedie, `format` ou `ci` n'est visible

4. Ce qui est deja sain

Ces points ne suffisent pas a compenser les risques, mais ils montrent que le projet a de bonnes bases.

- le socle moderne React 19 + TypeScript + Vite est present
- le projet utilise deja un linter et un compilateur strict
- une vraie separation existe entre pages, composants, services, config, types et templates
- les scripts SQL sont versionnes dans le depot
- un design system commence a exister avec `tokens.css`
- l'application couvre deja des cas metier concrets, pas seulement une maquette

5. Constats confirmes sur les parcours critiques

5.1 Conversion et confiance

C1. Le formulaire pre-session landing-solo contient une incoherence email reelle

Constat confirme :

- le haut du formulaire lit `responses.email`
- la validation landing-solo attend `responses.clientEmail`
- la zone de questions landing-solo utilise ensuite `responses.clientEmail`
- le bouton de soumission regarde encore `responses.email`

- la sauvegarde finale bascule vers `clientEmail`

Consequences metier :

- un prospect peut remplir un champ visible sans que toute la chaine lise la meme valeur
- la validation, le bouton et la persistance ne sont pas alignes

Impact :

- **eleve**

C2. Le formulaire contact n'a pas de canal fiable principal

Constat confirme :

- `ContactPage.tsx` envoie vers Crisp cote navigateur
- si Crisp n'est pas charge, le fallback est surtout un `console.log`
- le message de succes est pilote par la presence de Crisp et un `setTimeout`

Consequences metier :

- un message client peut etre perdu
- l'utilisateur peut croire que le message est parti alors que rien de fiable n'est stocke

Impact :

- **eleve**

C3. La page de confirmation affiche un succes sans relecture fiable cote serveur

Constat confirme :

- `ConfirmationPage.tsx` lit `paymentId`, `orderId` et `source` depuis l'URL
- elle reconstruit ensuite l'etat via `localStorage`
- elle affiche des messages tres rassurants avant verification fraiche de la commande

Consequences metier :

- un succes peut etre affiche trop tot
- la preuve de confirmation repose trop sur le navigateur

Impact :

- **eleve**

C4. La validation de paiement reste trop decidee par le front

Constat confirme :

- `OrderValidation.tsx` appelle `updateOrderStatus`
- ce composant peut mettre `payment_status = paid` depuis le navigateur
- `supabase.ts` expose directement l'update de la table `orders`

Consequences metier :

- le client ne doit pas pouvoir etre la source finale de validation du paiement
- la logique de confiance est du mauvais cote

Impact :

- **critique**

5.2 Securite et donnees

C5. Plusieurs scripts SQL documentent des politiques trop ouvertes

Constat confirme :

- `create_pre_session_forms.sql` ouvre la lecture publique avec `USING (true)`
- `create_live_sessions.sql` ouvre toutes les operations avec `FOR ALL USING (true)`
- la variante `pre_session_forms_migration.sql` donne les droits a tout utilisateur authentifie sans role admin plus fin

Consequences metier :

- lecture ou modification trop large possible selon le contexte reel de deploiement
- forte dependance a des regles non visibles en dehors du depot

Impact :

- **critique**

C6. Des donnees utiles mais sensibles restent exposees dans le navigateur

Constat confirme :

- `ConfirmationPage.tsx` lit `pendingOrder` dans `localStorage`
- certains identifiants remontent dans l'URL
- `supabase.ts` diffuse des `publicUrl` pour les images du bucket

Consequences metier :

- la surface d'exposition est plus large que necessaire
- la confidentialite minimale des donnees client est fragilisee

Impact :

- **eleve**

5.3 Maintainabilite et capacite d'evolution

C7. La dette type et lint est lourde sur les zones business

Constat confirme :

- 220 problemes lint
- tres nombreux `any`
- plusieurs `useEffect` avec dependances fragiles
- variables inutilisees
- interfaces trop larges ou trop floues dans `orders.ts` et `preSession.ts`

Impact :

- **eleve**

C8. Les gros ecrans concentrent trop de responsabilites

Constat confirme :

- `ConfiguratorPage.tsx` fait 6963 lignes
- `FormulairePréSession.tsx` fait 2139 lignes
- ces fichiers melangent rendu, validation, logique metier, persistence, navigation et etat local

Impact :

- **eleve**

C9. Il n'y a pas de filet de securite automatise visible

Constat confirme :

- pas de script de test
- pas de tests unitaires ni parcours critiques releves

Impact :

- **moyen a eleve**

5.4 Performance et structure

C10. Le chargement initial est trop large

Constat confirme :

- `App.tsx` importe en statique home, solutions, contact, configurateur, admin, confirmation et templates
- aucun chargement differe par route n'a ete releve
- Crisp est injecte globalement

Impact :

- **eleve**

C11. Les polices et certains assets sont charges de maniere peu rigoureuse

Constat confirme :

- Montserrat est chargee dans `index.html`
- Montserrat est rechargee dans `tokens.css`
- le systeme de tokens declare plusieurs variables en doublon ou redefinies

Impact :

- **moyen a eleve**

5.5 Qualite UI, accessibilite et SEO

C12. La navigation interne ne suit pas toujours les conventions SPA

Constat confirme :

- `Button.tsx` renvoie un `<a href>` pour la navigation interne
- cela contourne la navigation applicative de React Router

Impact :

- **moyen**

C13. L'accessibilite est presente par endroits mais reste inegale

Constat confirme :

- la page contact utilise surtout des placeholders sans labels visibles
- plusieurs modales et cartes ont deja ete signalees dans les audits existants
- la base est partielle mais pas harmonisee

Impact :

- **moyen a eleve**

C14. Le socle SEO reste minimal

Constat confirme :

- un seul `title` global dans `index.html`
- pas de meta description par page
- pas de `robots.txt`
- pas de `sitemap.xml`

Impact :

- **moyen**

6. Audit du respect des bonnes pratiques de developpement

Lecture "tech lead" : le projet respecte les bonnes pratiques de base d'un projet web moderne, mais il s'eloigne nettement des bonnes pratiques de robustesse sur les flux business.

6.1 Architecture et separation des responsabilites

Etat :

- **partiellement respecte**

Points sains :

- dossiers `pages` , `components` , `services` , `config` , `types` , `templates`

Ecart confirmés :

- composants trop grands
- logique metier, rendu et acces donnees parfois melanges dans le meme fichier
- `supabase.ts` cumule configuration, acces CRUD, debug et upload

Lecture tech lead :

- la structure generale est bonne
- la discipline de decoupage ne suit plus sur les zones critiques

6.2 Typage et contrat de donnees

Etat :

- **insuffisant sur les parcours critiques**

Points sains :

- TypeScript strict est active
- des interfaces de base existent

Ecart confirmés :

- `any` nombreux dans les types et composants
- `responses: Record<string, any>` affaiblit la qualite du pre-session
- `OrderRecord` transporte plusieurs blobs JSON non types

Lecture tech lead :

- le projet a active le bon outil mais ne l'exploite pas encore jusqu'au bout

6.3 Tests et prevention des regressions

Etat :

- **non respecte a un niveau suffisant**

Ecart confirmés :

- aucun script de test
- aucun test visible sur les flux critiques

Lecture tech lead :

- sans filet automatise, chaque correction business reste risque

6.4 Securite applicative

Etat :

- **insuffisant sur la source de confiance**

Points sains :

- Supabase et RLS sont utilises comme intention de socle

Ecart confirmé :

- paiement valide depuis le front
- politiques SQL trop ouvertes dans plusieurs scripts
- accès admin basé sur session visible plutôt que rôle métier confirmé
- données transitoires utiles stockées côté navigateur

Lecture tech lead :

- la sécurité est pensée, mais pas encore fermée

6.5 Gestion d'état et effets de bord

État :

- **fragile**

Ecart confirmé :

- plusieurs warnings `useEffect`
- dépendances manquantes
- reconstruction d'état depuis `URL + localStorage + state`

Lecture tech lead :

- cela crée des comportements difficiles à prévoir, tester et maintenir

6.6 Performance et chargement

État :

- **moyen**

Points sains :

- socle Vite moderne
- audits Lighthouse déjà disponibles

Ecart confirmé :

- pas de code splitting par route
- widget tiers chargé partout
- police chargée deux fois

Lecture tech lead :

- le projet n'est pas hors controle, mais il paie un surcout evitables des l'ouverture

6.7 Accessibilite et experience utilisateur

Etat :

- **partiellement respecte**

Ecart confirmés :

- labels manquants sur les formulaires
- navigation clavier et modales inegales
- liens et zones cliquables pas toujours semantiques

6.8 SEO technique

Etat :

- **encore debutant**

Ecart confirmés :

- metadonnees globales seulement
- pas de fichiers SEO de base

6.9 Design system et coherence front

Etat :

- **partiellement respecte**

Points sains :

- un systeme de tokens existe

Ecart confirmés :

- variables en doublon
- styles inline encore presents
- ecart visuel entre vitrine et admin

6.10 Outillage et hygiene d'equipe

Etat :

- **base presente, niveau equipe incomplet**

Points sains :

- lint en place
- TypeScript strict en place

Ecart confirmés :

- pas de CI visible
- pas de tests
- pas de format standard visible
- le lint existe mais l'etat reel du projet n'est pas maintenu propre

6.11 Configuration, secrets et valeurs en dur

Etat :

- **partiellement maitrise, mais trop expose cote maintenance**

Constats confirmés :

- les identifiants publics sont lus via `import.meta.env` dans [supabase.ts] (c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/config/supabase.ts) et [PayPalButton.tsx] (c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/components/PayPalButton.tsx)
- l'ID Crisp est en dur dans [App.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/App.tsx#L55)
- plusieurs valeurs métier et visuelles sont en dur dans les composants et services
- le fichier `.env` local du projet contient des valeurs réelles de configuration front

Lecture tech lead :

- utiliser des variables `VITE_*` pour des clés publiques front est normal
- en revanche, laisser des valeurs réelles dans `.env` et multiplier les constantes en dur rend la maintenance plus fragile

Ce qui est confirmé :

- **clés publiques front** : oui
- **secret serveur expose dans le code front** : non relevé
- **hygiène de configuration insuffisante** : oui

Ce qui est probable :

- le projet n'a pas encore de convention claire entre config centralisee et valeurs en dur

Ce qui reste a verifier :

- si des valeurs live equivalentes existent aussi dans d'autres fichiers non lus ici

6.12 Logs sensibles et exposition navigateur

Etat :

- **trop bavard en debug pour un flux business sensible**

Constats confirmes :

- [supabase.ts](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/config/supabase.ts) loggue des extraits de configuration, des payloads et des retours base
- [PayPalButton.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/components/PayPalButton.tsx) loggue le mode, la presence des variables, un apercu de `clientId` et l'URL de script
- [ContactPage.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/pages/ContactPage.tsx) loggue le contenu du formulaire en fallback
- [ConfirmationPage.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/pages/ConfirmationPage.tsx) depend de `localStorage`
- [supabase.ts](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/config/supabase.ts) expose des `publicUrl` pour les images

Lecture tech lead :

- on n'est pas face a une fuite critique de secret serveur relevee dans le front
- en revanche, le niveau de verbose debug est trop eleve pour des flux commande, paiement et contact

6.13 Incoherence de socle front

Etat :

- **confirme et structurant pour la maintenance**

Constats confirmés :

- [tokens.css](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/styles/tokens.css) charge Montserrat alors que [index.html](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/index.html) la charge déjà
- [tokens.css](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/styles/tokens.css) redefinit plusieurs familles de variables et certains noms se chevauchent
- [global.css](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/styles/global.css) definit `body` deux fois
- [Home.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/pages/Home.tsx) contient beaucoup de styles inline
- [ContactPage.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/pages/ContactPage.tsx) contourne aussi le systeme avec du style inline

Lecture tech lead :

- le design system existe
- mais il n'est pas encore la source de verite unique

6.14 Dette de coherence UI / design system

Etat :

- **moyenne a elevee**

Constats confirmés :

- [crispService.ts](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/services/crispService.ts) remet des couleurs en dur alors que le projet a des tokens
- [Footer.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/components/Footer.tsx) utilise encore des liens placeholders `#`
- [Button.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/components/Button.tsx) ne distingue pas navigation interne et lien externe

- la vitrine, les templates et l'admin ne semblent pas gouvernes par le meme niveau de regles visuelles

Ce qui est probable :

- plusieurs ecrans ont ete finalises localement sans passage systematique par une bibliotheque commune

6.15 Coherence visuelle et qualite d'integration

Etat :

- **partiellement confirme, a completer idealement par revue visuelle**

Constats confirmes dans le code :

- centrage, alignements et espacements sont souvent decides composant par composant via styles inline
- [Home.tsx](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/pages/Home.tsx) force localement `textAlign` , `margin` , `justifyContent` , `maxWidth`
- [global.css](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/styles/global.css) et [tokens.css](c:/Users/Mariedo/OneDrive/Documents/Dev/fsvb-studio-portfolio-analysis/fsvb-studio-portfolio/portfolio-app/src/styles/tokens.css) ne suffisent donc pas a garantir des compositions homogenes

Ce qui est probable :

- des ecarts visuels reels existent entre pages, templates et composants
- les objets, images ou blocs peuvent ne pas partager la meme logique de centrage

Ce qui reste a verifier :

- confirmation visuelle ecran par ecran sur desktop et mobile

6.16 Gouvernance du front

Etat :

- **insuffisante**

Constats confirmes :

- il existe bien un socle commun

- mais il n'existe pas encore de discipline visible qui impose l'usage prioritaire des tokens, classes, patterns de layout et regles de navigation

Lecture tech lead :

- le probleme n'est pas l'absence de socle
- le probleme est l'absence de gouvernance suffisante autour de ce socle

7. Hypotheses probables

H1. Le projet a grandi par ajouts successifs plus que par harmonisation globale

Probable, car :

- la structure generale est bonne
- mais les conventions deviennent moins stables dans les zones critiques

H2. La vitesse de livraison front a pris le pas sur la fermeture technique

Probable, car :

- plusieurs parcours sont fonctionnels en apparence
- mais la preuve finale reste cote navigateur

H3. Le configurateur agit comme centre de gravite de la dette

Probable, car :

- il concentre taille, logique, etat et warnings

8. Points a verifier avec l'environnement reel

- verification d'un vrai paiement de bout en bout
- verification des polices reellement deployees dans Supabase
- verification du role admin reel, pas seulement de la session visible
- verification du niveau de confidentialite attendu pour les images exposees
- verification de l'usage reel du bucket `project-images`

9. Methodes de resolution pas a pas

9.1 Corriger le pre-session landing-solo

Objectif :

une seule cle email du champ visible jusqu'a la sauvegarde.

Etapes :

1. choisir une seule cle canonique : `clientEmail`
2. remplacer tous les acces `responses.email` lies au pre-session landing-solo
3. aligner validation, champ, bouton et payload final
4. verifier la reprise des donnees existantes si certains brouillons utilisent encore `email`
5. ajouter un test simple "champ rempli -> validation -> sauvegarde"

9.2 Rendre le contact fiable sans dependance a Crisp

Objectif :

le message client doit partir dans un canal principal fiable.

Etapes :

1. definir un canal principal : table Supabase, email transactionnel ou fonction serveur
2. conserver Crisp comme canal secondaire, pas comme source de verite
3. afficher le message de succes seulement apres retour positif du canal principal
4. journaliser l'erreur et afficher un message clair si l'envoi echoue
5. ajouter un test nominal et un test "Crisp absent"

9.3 Fiabiliser la confirmation de commande

Objectif :

la page de succes doit relire une commande verifiee.

Etapes :

1. utiliser l'URL seulement pour porter un identifiant de reference
2. relire la commande depuis la base ou une fonction serveur
3. verifier statut commande et statut paiement
4. afficher un ecran d'attente tant que la verification n'est pas terminee
5. afficher un ecran d'erreur ou de support si la commande n'est pas retrouvee
6. vider les donnees transitoires navigateur une fois la verification terminee

9.4 Sortir la validation finale du paiement du navigateur

Objectif :

le front ne doit plus pouvoir marquer une commande comme payee.

Etapes :

1. creer une fonction serveur ou webhook de verification de paiement
2. faire remonter au serveur la preuve de paiement recue
3. laisser le serveur verifier la preuve puis mettre a jour `orders`
4. rendre le front purement declaratif : "j'attends le resultat"
5. tracer chaque changement de statut paiement
6. tester le cas nominal et le cas fraude ou preuve invalide

9.5 Reformer les politiques SQL

Objectif :

chaque table sensible doit avoir des droits limites au juste besoin.

Etapes :

1. lister table par table les usages reels : lecture publique, insertion publique, lecture admin, update admin
2. supprimer les politiques globales `USING (true)` ou `FOR ALL`
3. differencier client public, utilisateur authentifie et admin metier
4. verifier les acces storage avec la meme rigueur
5. tester chaque policy avec un compte public et un compte admin

9.6 Reprendre la dette type et lint

Objectif :

remonter rapidement le niveau de confiance du code.

Etapes :

1. traiter d'abord les fichiers critiques : `ConfiguratorPage.tsx` , `FormulairePréSession.tsx` , `OrderValidation.tsx` , `ConfirmationPage.tsx`
2. remplacer les `any` les plus structurants par des types reels
3. corriger les warnings `useEffect`
4. stabiliser les types partages dans `orders.ts` et `preSession.ts`

5. fixer un objectif par palier, par exemple passer de 220 problemes a moins de 80 puis a zero sur les flux critiques

9.7 Introduire un premier filet de tests

Objectif :

proteger les flux qui font vendre et encaissent.

Etapes :

1. choisir un socle simple : Vitest + React Testing Library
2. couvrir d'abord pre-session, contact, confirmation et validation commande
3. ecrire un test nominal et un test d'erreur par flux
4. brancher ces tests au lint et au build local
5. ajouter ensuite 1 ou 2 parcours E2E sur les moments les plus sensibles

9.8 Decouper progressivement le configurateur

Objectif :

retirer la concentration de risque d'un seul fichier.

Etapes :

1. cartographier ses blocs : donnees, navigation, preview, paiement, upload, modales
2. sortir d'abord les utilitaires purs et les hooks sans effet visuel
3. extraire ensuite les sections UI les moins couplees
4. terminer par la logique de paiement et de persistance
5. verifier a chaque extraction que le comportement visible ne change pas

9.9 Assainir configuration, valeurs en dur et identifiants tiers

Objectif :

avoir une seule politique claire pour ce qui doit etre centralise, parametre ou public.

Etapes :

1. lister toutes les valeurs de service tiers et metier actuellement en dur
2. classer en trois familles : public front, config environnement, vrai secret serveur
3. sortir du code les valeurs qui relevent de la configuration
4. centraliser les constantes metier restantes dans un dossier de config clair

5. interdire qu'un vrai secret serveur soit utilise cote front

9.10 Reduire les logs sensibles

Objectif :

garder des logs utiles sans exposer inutilement configuration, donnees client ou details de paiement.

Etapas :

1. recenser les `console.log` , `console.warn` et `console.error` des flux contact, commande et paiement
2. supprimer les logs de payload, d'extraits de cles et d'URL sensibles
3. garder seulement des messages techniques courts, sans donnees client
4. limiter les logs detaillees au dev local via un garde-fou centralise

9.11 Reprendre la gouvernance du front

Objectif :

faire du design system la reference par default, et non une option parmi d'autres.

Etapas :

1. fixer une hiérarchie simple : tokens, classes utilitaires, composants, exceptions
2. supprimer les doublons de polices et de variables dans les styles globaux
3. reduire les styles inline sur les pages les plus visibles
4. clarifier les regles de couleur, typo, espacements, bordures et ombres
5. documenter ce qui est autorise et ce qui ne l'est plus

9.12 Reprendre la coherence visuelle et l'integration

Objectif :

avoir des pages, composants et templates qui paraissent venir du meme produit.

Etapas :

1. faire une revue visuelle desktop et mobile des pages publiques, templates et admin
2. lister les ecarts de centrage, marges, tailles et alignements
3. corriger d'abord les composants communs qui propagent ces ecarts
4. harmoniser ensuite les pages les plus visibles

5. valider a la fin avec une petite grille de controle UI commune

10. Plan de correction coherent

Lot 1. Securiser confiance et conversion

Actions :

1. corriger l'incoherence `email/clientEmail`
2. brancher le contact sur un canal fiable principal
3. relire la commande avant tout succes complet
4. sortir la validation finale du paiement du front
5. refermer les policies les plus ouvertes

Lot 2. Stabiliser le socle de developpement

Actions :

1. reprendre les types partages
2. corriger les `any` et `useEffect` les plus risquants
3. poser les premiers tests
4. clarifier le role admin
5. reduire l'exposition des donnees navigateur

Lot 3. Faire monter la qualite globale

Actions :

1. charger les grosses routes a la demande
2. nettoyer la strategie police et assets
3. poser le SEO par page
4. harmoniser accessibilite et design system
5. decouper progressivement le configurateur

11. Priorisation finale

A faire tout de suite

- validation paiement

- confirmation fiable
- contact fiable
- pre-session landing-solo
- politiques SQL

A faire juste apres

- dette lint et types
- acces admin
- premiers tests
- reduction des donnees dans le navigateur

A faire ensuite

- code splitting
- SEO technique
- accessibilite visible
- harmonisation design system
- refactor progressif du configurateur

12. Conclusion simple

Le projet a deja une vraie valeur.

Le diagnostic "tech lead" ne dit pas de repartir de zero.

Il dit plutot :

1. remettre la confiance du bon cote
2. fermer les ouvertures les plus visibles
3. redonner de la lisibilite au code
4. installer enfin un cadre de qualite durable

Le point encourageant reste net :

- les problemes majeurs sont confirmes
- ils sont localises
- ils peuvent etre traites par ordre

13. Preuves principales

- `fsvb-studio-portfolio/portfolio-app/src/App.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/pages/ContactPage.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/pages/ConfirmationPage.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/components/OrderValidation.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/components/FormulairePréSession.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/config/supabase.ts`
- `fsvb-studio-portfolio/portfolio-app/src/types/orders.ts`
- `fsvb-studio-portfolio/portfolio-app/src/types/preSession.ts`
- `fsvb-studio-portfolio/portfolio-app/src/styles/tokens.css`
- `fsvb-studio-portfolio/portfolio-app/src/components/Button.tsx`
- `fsvb-studio-portfolio/portfolio-app/src/pages/AdminDashboard.tsx`
- `fsvb-studio-portfolio/portfolio-app/index.html`
- `fsvb-studio-portfolio/portfolio-app/sql/create_pre_session_forms.sql`
- `fsvb-studio-portfolio/portfolio-app/sql/pre_session_forms_migration.sql`
- `fsvb-studio-portfolio/portfolio-app/sql/create_live_sessions.sql`
- `npm run lint`